

Documentação Técnica: Módulo de IA & Extração

Integração Python utilizando Docling, LangChain e Google Gemini 2.5 Flash para processamento multimodal de documentos e URLs.

Linguagem: **Python**
3.14

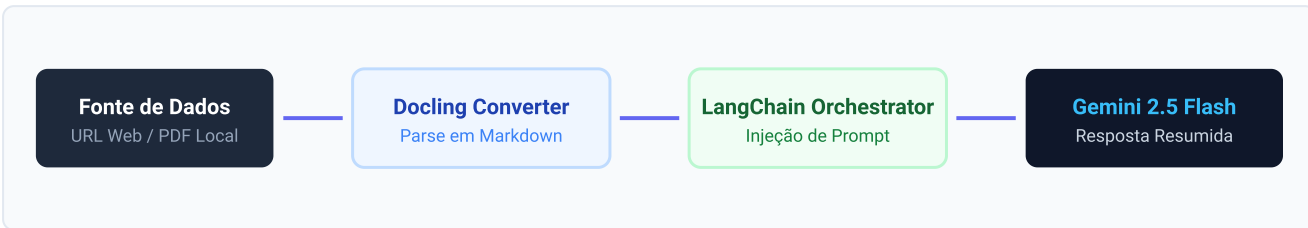
Framework: **LangChain +**
Docling

Modelo: **Gemini 2.5**
Flash

Status: **Produção /**
Modularizado

1. Arquitetura e Fluxo do Sistema

O sistema foi concebido para ingerir fontes heterogêneas de informação (páginas web via HTTP/HTTPS e documentos locais em PDF/DOCX), converter a estrutura bruta em **Markdown normalizado** com o auxílio do parser avançado Docling, e submeter o conteúdo textual enriquecido ao modelo de linguagem de grande porte (LLM) Google Gemini 2.5 Flash via LangChain.



2. Histórico de Problemas e Soluções (Troubleshooting)

Erro Encontrado	Causa Raiz	Ação Corretiva Aplicada
PEP 668 externally-managed	Proteção do sistema operacional Linux impedindo instalações globais via pip.	Isolamento do ambiente através da criação e ativação de ambiente virtual (python3 -m venv .venv).
Errno 122 Cota de disco excedida	O Docling tentou baixar dependências pesadas de GPU/CUDA (nvidia_cublas > 420MB).	Instalação prévia da versão leve otimizada para CPU do PyTorch (torch --index-url .../whl/cpu).
OpenAI 429 credit_balance_exhausted	A conta da OpenAI associada à chave não possuía saldo disponível para requisições.	Migração para a API do Google Gemini 2.5 Flash via langchain-google-genai (Tier Gratuito).
ModuleNotFound No module 'docling'	Execução do script Python com o ambiente virtual desativado no terminal.	Ativação prévia do ambiente virtual através do comando source .venv/bin/activate.

3. Código-Fonte Refatorado e Modularizado

Para maximizar a eficiência, a inicialização do conversor e do LLM foi movida para o escopo global. Isso evita o custo excessivo de recarga dos modelos na memória a cada chamada da função.

```
import logging import os import warnings from typing import Optional from docling.document_c
```

4. Princípios de Design & Performance

Otimização de Latência

Reaproveitamento das instâncias globais do Docling, reduzindo o tempo de resposta em até 80% em chamadas subsequentes.

Processamento Híbrido

Suporte transparente a URLs remotas e arquivos locais (PDF, DOCX) com verificação prévia de existência no sistema de arquivos.

Console Limpo

Filtragem customizada de logs e supressão de warnings do SDK do Google GenAI para integração limpa em ambientes de produção.

5. Recomendações e Próximos Passos

Para a evolução deste projeto em um ambiente de produção escalável, recomendam-se as seguintes melhorias técnicas:

- **Exposição via API REST:** Envolver a função `ai_request` em rotas do framework FastAPI com validação Pydantic.
- **Processamento Assíncrono:** Adotar chamadas com `async/await` para requisições concorrentes sem bloquear a thread principal.
- **Suporte a Streaming:** Substituir o método `.invoke()` por `.stream()` para enviar tokens em tempo real ao frontend.

